

Codity Assignment Submission - Job Scheduler & Background Worker

1. Project Setup & Organization

This project is built using a modern, decoupled architecture:

- **Backend:** FastAPI with Python 3.10+, utilizing async programming (`asyncio` , `asyncpg`).
- **Database:** PostgreSQL (via SQLAlchemy 2.0 Async engine). We utilize Alembic for migrations (if needed) but primarily rely on `init_db.py` for bootstrapping the schema.
- **Worker:** A dedicated headless python worker process (`app/worker.py`) that continuously polls the database for `QUEUED` jobs, executes them concurrently using an `asyncio.Semaphore` , and handles failures/retries.
- **Frontend:** A React 19 application built with Vite, TypeScript, TailwindCSS, and Lucide Icons for a beautiful, responsive dashboard.

2. Database Design & ORM

Our PostgreSQL schema is fully normalized and leverages relationships carefully:

- **Users Table:** Stores `admin` credentials (JWT authenticated routes).
- **Queues Table:** Defines job queues (e.g., `Default Queue`), specifying `retry_strategy` (fixed/exponential) and `max_retries` .
- **Jobs Table:** Tracks all jobs submitted to the system. Stores `payload` , `status` (`QUEUED` , `CLAIMED` , `RUNNING` , `COMPLETED` , `FAILED`), `attempts` , `max_retries` , and timestamp fields.
- **Job Logs Table:** Replaces the basic `error_summary` string with a fully relational log table to track execution history, preserving every attempt and failure reason.

- **Concurrency:** We rely on PostgreSQL's `SELECT ... FOR UPDATE SKIP LOCKED` (implemented via SQLAlchemy's `with_for_update(skip_locked=True)`) to safely claim jobs and completely prevent race conditions when running multiple parallel workers.

3. Backend API Implementation

The backend exposes a fully RESTful API with validation powered by Pydantic.

- **Authentication:** `POST /auth/token` provides JWT tokens. All sensitive API routes (queue creation, job creation, DLQ management) are protected by standard OAuth2 Bearer token dependencies.
- **Core Operations:** Endpoints to create queues, list queues, insert jobs, and fetch job statuses.
- **Pagination:** Implemented cursor/offset pagination for listing jobs within a queue (`limit` , `offset` in `GET /jobs/queue/{queue_id}`).
- **Logs & Retry APIs:** Added dedicated endpoints to fetch execution logs (`GET /jobs/{job_id}/logs`) and explicitly retry a dead-lettered job (`POST /jobs/{job_id}/retry`).

4. The Worker Component

The heart of the asynchronous processing:

- **State Machine:** Jobs move strictly from `QUEUED` -> `CLAIMED` -> `RUNNING` -> `COMPLETED` (or `FAILED`).
- **Retry Logic:** If a job fails, the worker reads the Queue's `retry_strategy` and schedules the job's `scheduled_at` timestamp into the future.
- **Dead Letter Queue (DLQ):** Once a job exhausts its `max_retries` , it lands in the `FAILED` state permanently and becomes visible in the DLQ view on the dashboard.
- **Simulated Delays:** Included a `test_workflow.py` script that proves the worker appropriately handles mocked payload delays and handles forced failures perfectly.

5. Frontend Dashboard & UX

A pristine, developer-friendly dashboard that visualizes the state of the system in real time:

- **Overview Tab:** Live view of queues, job throughput, status breakdowns, and deep dives into specific jobs (with an expandable execution logs terminal).
- **Queues Tab:** Allows manual creation of new Queues with configurable retry strategies.
- **Dead Letter Tab:** Dedicated space to review permanently failed jobs with a 1-click **Retry** button that resets the job's attempts and places it back in the `QUEUED` state.
- **Workers Tab:** Real-time monitoring showing live polling worker nodes and their last heartbeat.

6. Bonus Implementations Completed

- **AI Failure Analysis (Gemini):** When a job exhausts all retries and lands in the DLQ, the worker triggers an asynchronous call to the `gemini-2.5-flash` model via the official `@google/genai` SDK. The AI analyzes the `error_summary` and the `payload`, generating an intelligent natural-language `ai_failure_summary` surfaced directly in the dashboard UI.
- **100% Test Coverage on Concurrency:** A rigorous `pytest` suite running 12 separate tests, guaranteeing that locking logic `skip_locked=True` operates perfectly without deadlocking or double-executing jobs.

Running the Application

1. Database & Backend

```
cd backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
set DATABASE_URL=postgres+asyncpg://jobuser:jobpassword@localhost:5432/job
```

```
set SYNC_DATABASE_URL=postgresql://jobuser:jobpassword@localhost:5432/job_s
python init_db.py
uvicorn app.main:app --reload
```

2. Start the Worker (New Terminal)

```
cd backend
.\venv\Scripts\activate
set DATABASE_URL=postgresql+asyncpg://jobuser:jobpassword@localhost:5432/jc
set GEMINI_API_KEY=your_key_here
python -m app.worker
```

3. Start Frontend (New Terminal)

```
cd frontend
npm install
npm run dev
```

Run `python test_workflow.py` in the backend to inject sample data and see the system come alive!